



Basics of Leo and Aleo

A learning group for ZK and SNARK application development

Daniel Szego

In: <https://www.linkedin.com/in/daniel-szego/>



Respectful collaboration starts here

**ALL ARE
WELCOME**

**TODO
BIENVENIDOS**

**みなさん
歓迎します。**

www.lfdecentralizedtrust.org/code-of-conduct

Agenda



- *Logistics*
- *Zero knowledge proofs - summary*
- *Classical zkSNARK flow*
- *Leo programming language and framework*
- *Leo language elements*
- *Leo program structure*
- *Aleo*
- *Aleo building blocks*
- *Demo, hello world*
- *Challenge*
- *Links, Resources, Literature*
- *Q&A*

Logistics



Every month or two months

Similar structure, similar logistics + **guest lectures**

Theory:

- Look tables,
- Zk variations: STARKs, FRI
- recursive SNARKs

Programming / platforms:

- ZoKrates, Leo
- Zinc / ZKSync

Applications:

- ZK bridges
- ZK Virtual Machines
- ZK Identity / verifiable credential - presentations
- Voting



Discord channel:

<https://discord.com/channels/905194001349627914/1329201532628898036>

Meetup.com:

<https://www.meetup.com/lfdt-hungary/events/305634614/>

Repo with all the contents:

<https://github.com/LF-Decentralized-Trust-labs/zk-learning-group>

Zero knowledge proofs - summary

Verifiable computation with cryptographic proof

Scarce resource:

- limited verification time: no re-execution
- storage limits: smaller proof
- limited information

Computation: arithmetic circuit : $C(x, w) \rightarrow F$

- x public input
- w private input, witness
- high level computation
- arithmetic circuit
- polynomials

Prover algorithm: $P(pp, x, w) \rightarrow \text{proof}$

Verifier algorithm: $V(vp, x, \text{proof}) \rightarrow \text{accept / reject}$

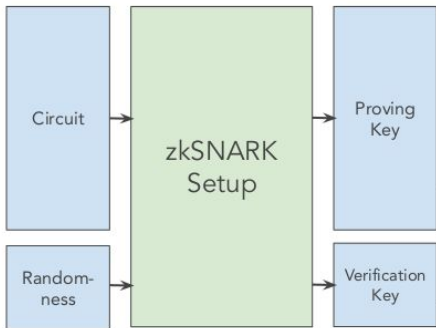
Zero Knowledge Proof



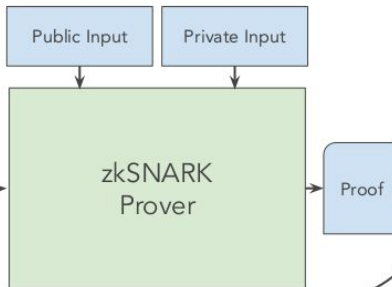
Interactive / non-interactive proofs

Classical zkSNARK flow

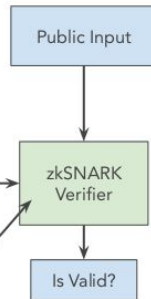
1. a one-time setup is run, producing a reusable proving and verification key



2. a prover generates a zero-knowledge proof for some public and private inputs



3. a verifier outputs a decision bit



Leo programming language and framework

Syntax is inspired heavily by rust and javascript

General-purpose, zero-knowledge programming language

Turing complete to produce state transaction to ZK applications

Testing framework,

Package registry,

Import resolver,

Remote compiler,

Theorem prover

Formal verification: proving execution correctness



<https://www.leo-lang.org/>

Leo language elements

Types:

- Scalar: field, group element, integer, boolean, addresses
- Aggregate: :tuples, arrays, circuits (class)
- Special types: Signatures, identifiers,

State:

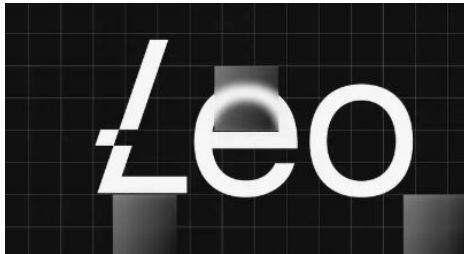
- Public: records
- Private: mapping

Control structures:

- If-then
- for loop

Functions

Interfaces / interface implementations



<https://docs.zkproof.org/pages/standards/accepted-workshop4/proposal-leo.pdf>

Leo program structure

Programs

Constructor

Imports

Structs

Records

Mappings

Interfaces

Functions

Interface implementations

```
import foo.aleo;

const F00: u64 = 1u64;

struct Message {
  sender: address,
  object: u64,
}

fn compute(a: u64, b: u64) -> u64 {
  return a + b + F00;
}

program hello.aleo {
  mapping account: address => u64;

  record Token {
    owner: address,
    amount: u64,
  }

  fn mint_public(
    public receiver: address,
    public amount: u64,
  ) -> (Token, Final) {
    let token: Token = Token { owner: receiver, amount };
    return (token, final {
      let current_amount: u64 = Mapping::get_or_use(account, receiver, 0u64);
      Mapping::set(account, receiver, current_amount + amount);
    });
  }

  @noupgrade
  constructor() {}
}
```

<https://docs.leo-lang.org/>

Aleo

L1 blockchain

Strong privacy focus

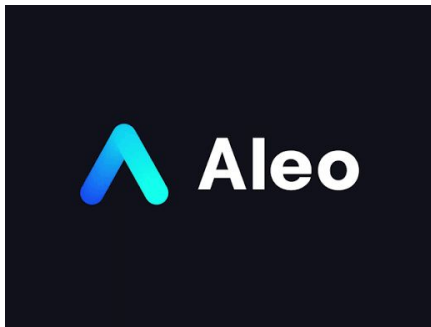
UTXO (Unspent Transaction Output) based

General smart contract programmability

Smart contracts with Leo: privacy + verifiability

Hybrid consensus:

- Proof of Succinct Work (PoSW): provers generate and submit proofs
- AleoBFT, a Byzantine fault-tolerant consensus mechanism based on the Narwhal + Bullshark stack.



<https://aleo.org/>

Aleo building blocks

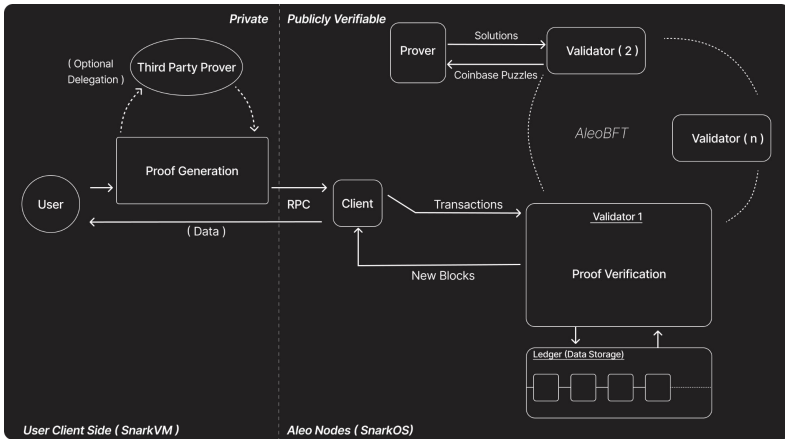
Off-chain execution

snarkVM

Validators: produce blocks and verify zero knowledge proofs

Provers: contribute computational power to the Aleo network by solving synthesis puzzles Proof of Succinct Work (PoSW)

Aleo token: reward, staking



Demo



Hello Leo

https://docs.leo-lang.org/getting_started/hello

Challenge



**Build a simple (9,9)
sudoku puzzle with
Leo**

Links, Resources, Literature



Leo Language

<https://www.leo-lang.org/>

LEO: A Programming Language for Formally Verified, Zero-Knowledge Applications

<https://docs.zkproof.org/pages/standards/accepted-workshop4/proposal-leo.pdf>

Aleo

<https://aleo.org/>

Aleo documentation

<https://docs.aleo.org/>

Hello Leo

https://docs.leo-lang.org/getting_started/hello



Happy Hunting for the SNARK :)

Q & A

Daniel Szego

In: <https://www.linkedin.com/in/daniel-szego/>

